第三章作业

1

给定一个只保存 `int` 的栈 `S`，原有基本操作如下：

- `S.push(x)`：将 `x` 压入栈中。
- `S.pop()`：将栈顶元素出栈，并返回该元素。
- `S.top()`：返回栈顶元素。

请在不增加新的容器类数据结构的前提下，对上述操作进行必要的重写，并增加：

- `S.getMax()`：返回栈中的最大元素。

要求所有操作的时间复杂度均为 $O(1)$。可以在原栈中额外压入辅助整数；无需考虑栈空或栈满。请给出各操作的伪代码，并简要说明辅助信息的含义。

伪代码：

```
push(x):
   if S.empty():
      S.push(x)
      S.push(x)
   else:
      maxVal = S.top()
      S.push(x)
      S.push(max(x, maxVal))
pop():
   S.pop()
   return S.pop()
top():
   maxVal = S.pop()
   actualVal = S.top()
   S.push(maxVal)
   return actualVal
getMax():
   return S.top()
```

思路讲解：在每个元素入栈时，同时将 stack 当前的最大值也压入栈中，这样在出栈时可以同时弹出当前元素和对应的最大值，从而保证 `getMax()` 操作可以在 $O(1)$ 时间内完成。

2

由 n 对左右括号组成的字符串中，若从左到右扫描时任意前缀内左括号数都不少于右括号数，且最终两者数量相等，则称为合法括号序列。

请回答长度为 $2n$ 的合法括号序列共有多少种，并写出递推关系、初始条件以及通项公式。另列出 $n = 3$ 时的全部合法序列。

```cpp
#include<iostream>
#include<vector>
using namespace std;

int main(){
    int n; cin >> n;
    vector<vector<int>> dp(n+1, vector<int>(n+1, 0));
    dp[0][0] = 1;
    for(int i = 0; i <= n; i++){
        for(int j = 0; j <= i; j++){
            if(i > 0) dp[i][j] += dp[i-1][j];
            if(j > 0) dp[i][j] += dp[i][j-1];
        }
    }
    cout << dp[n][n] << endl;
    return 0;
}
```

其中 dp[i][j] 表示使用 i 个左括号和 j 个右括号组成的合法序列数。

```
((()))
(()())
(())()
()(())
()()()
```

3

浏览器使用两个栈实现“后退”和“前进”：

- 栈 `Back` 保存可后退的页面；栈 `Forward` 保存可前进的页面。
- 变量 `current` 表示当前页面；访问新页面时，原当前页面压入 `Back`，并清空 `Forward`。
- `back()`（后退）：若 `Back` 非空，将 `current` 压入 `Forward`，再弹出 `Back` 的栈顶作为新的 `current`；否则不操作。
- `forward()`（前进）：若 `Forward` 非空，将 `current` 压入 `Back`，再弹出 `Forward` 的栈顶作为新的 `current`；否则不操作。

请回答：

1. 仅用 `push`、`pop`、`empty` 描述 `visit(p)`、`back()` 和 `forward()`。
2. 初始时 `current = Home`，两个栈均为空，依次执行：

   ```
   visit(A), visit(B), back(), visit(C), back(), forward(), back()
   ```

   写出最后的 `current`，以及两个栈中从栈底到栈顶保存的页面。

```
stack<T> Back, Forward;
void visit(T p) {
   Back.push(current);
   current = p;
   while(!Forward.empty()) Forward.pop();
}
void back() {
   if(!Back.empty()) {
      Forward.push(current);
      current = Back.top();
      Back.pop();
   }
}
void forward() {
   if(!Forward.empty()) {
      Back.push(current);
      current = Forward.top();
      Forward.pop();
   }
}
```

最后的 current = A，Back 栈中从栈底到栈顶为 Home，Forward 栈中从栈底到栈顶为 C。

4

已知栈的三个基本操作定义如下：

- `push(S, x)`：将元素 `x` 压入栈 `S`。
- `pop(S)`：弹出并返回栈 `S` 的栈顶元素。
- `isEmpty(S)`：判断栈 `S` 是否为空。

请设计一种方法，用两个普通栈 `S1`、`S2` 实现队列的三个基本操作：

- `enQueue(x)`：将元素插入队尾。
- `deQueue()`：删除并返回队首元素。
- `isEmpty()`：判断队列是否为空。

要求：

1. 说明两个栈各自保存什么，并写出 `enQueue` 与 `deQueue` 的伪代码。
2. 分析单次操作的最坏时间复杂度。
3. 若依次执行以下操作序列：

   ```
   enQueue(5), enQueue(8), deQueue(), enQueue(2), enQueue(7), deQueue(),
   deQueue()
   ```

   请写出每次 `deQueue()` 的返回结果，并给出操作结束后队列中从队首到队尾的元素。

```
stack<int> S1, S2;
void enQueue(int x) {
   S1.push(x);
}
int deQueue() {
   if(S2.empty()) {
      while(!S1.empty()) {
         S2.push(S1.top());
         S1.pop();
      }
   }
   return S2.pop();
}
bool isEmpty() {
   return S1.empty() && S2.empty();
}
```

S1 和 S2 二者整体保存队列的元素，S1 用于入队，S2 用于出队。具体来讲就是，每次要出队是，要出的元素在 S1 的栈底，所以需要将 S1 的元素全部倒入 S2 中，这样 S2 的栈顶就是队首元素。

deQueue 单次操作的最坏时间复杂度为 O(n)，其中 n 为当前队列中元素的数量。其余操作的时间复杂度为 O(1)。

deQueue() 的返回结果依次为 5, 8, 2，操作结束后队列中从队首到队尾的元素为 7。